JOMO KENYATTA UNIVERSITY
OF
AGRICULTURE & TECHNOLOGY

SCHOOL OF OPEN, DISTANCE AND eLEARNING
P.O. Box 62000, 00200
Nairobi, Kenya
E-mail: elearning@jkuat.ac.ke

ICS 2302 SOFTWARE ENGINEERING 1

This presentation is intended to covered within one week. The notes, examples and exercises should be supplemented with a good textbook. Most of the exercises have solutions/answers appearing elsewhere and accessible by clicking the green Exercise tag. To move back to the same page click the same tag appearing at the end of the solution/answer.

Errors and omissions in these notes are entirely the responsibility of the author who should only be contacted through the **Department of Curricula & Delivery (SODeL)** and suggested corrections may be e-mailed to elearning@jkuat.ac.ke.

LESSON 1

Software Engineering

Learning Outcomes

A study of this lesson should enable students to:

- Describe what is Software engineering
- Reflect on different software engineering methodologies

1.1. Introduction

The term *software engineering* first appeared in the 1968 NATO Software Engineering Conference and was meant to provoke thought regarding the current "software crisis" at the time. Since then, it has continued as a profession and field of study dedicated to creating software that is of higher quality, more affordable, maintainable, and quicker to build. Since the field is still relatively young compared to its sister fields of engineering, there is still much work and debate around what software engineering actually is, and if it deserves the title engineering. It has grown organically out of the limitations of viewing software as just programming. Software development is a term sometimes preferred by practitioners in the industry who view software engineering as too heavy handed and constrictive to the malleable process of

◀◀ ▶▶
◀ ▶
◀ Doc Doc ▶
Back Close

creating software. Yet, in spite of its youth as a profession, the field's future looks bright as Money Magazine and Salary.com rated software engineering as the best job in America in 2006.

1.1.1. Definition

Software engineering is the application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software, and the study of these approaches; that is, the application of engineering to software.

Further Definitions

What is the difference between software engineering and computer science?

Essentially, computer science is concerned with the theories and methods which underlie computers and software systems whereas software engineering is concerned with the practical problems of producing software.

System Engineering

System Engineering or more precisely computer based system engineering is concerned with all aspects of the development and evolution of complex systems where software plays a major role. System engineering is therefore concerned with hardware development policy and process design and system development

as well as software engineering.

Software Process

A software process is a set of activities and associated results which produce a software product. These activities are mostly carried out by software engineers. These activities are:-

- Software specifications – the functionality of the software and constrains on its operation must be defined.
- Software development – The software to meet the specification must be produced.
- Software validation – the software must be validated to ensure that it does what the customer wants.
- Software evolution – the software must evolve to meet changing customer needs.

◀◀ ▶▶ ◀ ▶ ◀ Doc Doc ▶ Back Close

Software process descriptions

When we describe and discuss processes, we usually talk about the activities in these processes such as specifying a data model, designing a user interface, etc. and the ordering of these activities. Process descriptions may also include:

- Products, which are the outcomes of a process activity
- Roles, which reflect the responsibilities of the people involved in the process
- Pre- and post-conditions, which are statements that are true before and after a process activity has been enacted or a product produced.

Plan-driven and agile processes

- Plan-driven processes are processes where all of the process activities are planned in advance and progress is measured against this plan.
- In agile processes, planning is incremental and it is easier to change the process to reflect changing customer requirements.
- In practice, most practical processes include elements of both plan-driven and agile approaches.
- There are no right or wrong software processes

1.2. Software Process Model

A software process model is a simplified description of a software process which is presented from a particular perspective. Process model may include activities which are part of the software process, software products and roles of people involved in software engineering. Some examples of the types of software process model which may be produced are;-

- A workflow Model – This shows the sequence of activities in the process along with their inputs, outputs and dependencies. The activities in this model represent human actions.
- A data flow or activity model – this represents the process as a set of activities each of which carries out some data

transformation. It shows how the input to the process such as a specification is transformed to an output such as design. The activities here may be at a lower level than activities in a workflow model. They may represent transformations carried out by people or by computers.

- A role model/action model - This represents the roles of the people involved in the software process and the activities for which they are responsible.

1.3. Software Engineering Methods

A software engineering method is a structured approach to software development whose aim is to facilitate the production of high quality software. In a cost effective objected oriented etc. All methods are based on the idea of developing models of a

system which may be represented graphically and using these models as a system specification or design.

Case

The acronym **CASE** stands for Computer Aided Software Engineering. It covers a wide range of different types of program which are used to support software process activities such as requirements analysis, system modeling, debugging and testing.

1.3.1. Attributes of good software

As well the services which they provide, software products have a number of other associated attributes which reflect the quality of that software. These attributes are not directly concerned with what that software does. Rather, the reflection of the source

◀◀ ▶▶ ◀ ▶ ◀ Doc Doc ▶ Back Close

program and associated documentation. They include:-

- Maintainability – software should be written in such a way that it may evolve to meet the changing needs of customers. This is a critical attribute because software change is an inevitable requirement of a changing business environment.
- Dependability – software dependability has a range of characteristics including reliability, security and safety. Dependable software should not cause physical or economic damage in the event of system failure. Malicious users should not be able to access or damage the system.
- Efficiency – software should not make wasteful use of system resources such as memory and processor cycles. Efficiency therefore includes responsiveness, processing time,

memory utilisation, etc.

- Usability – software must be usable, without undue effort, by the type of user for whom it is designed. This means that it should have an appropriate user interface and adequate documentation, it must be understandable, usable and compatible with other systems that they use.

Example ✎. What differences has the web made to software engineering?

Solution:

The web has led to the availability of software services and the possibility of developing highly distributed service-based systems. Web-based systems development has led to important advances in programming languages and software reuse.

◀◀ ▶▶ ◀ ▶ ◀ Doc Doc ▶ Back Close

Example ✎**.** What are the best software engineering techniques and methods?

Solution:

While all software projects have to be professionally managed and developed, different techniques are appropriate for different types of system. For example, games should always be developed using a series of prototypes whereas safety critical control systems require a complete and analyzable specification to be developed. You can't, therefore, say that one method is better than another.

1.3.2. Software engineering fundamentals

Some fundamental principles apply to all types of software system, irrespective of the development techniques used:

- Systems should be developed using a managed and understood development process. Of course, different processes are used for different types of software.
- Dependability and performance are important for all types of system.
- Understanding and managing the software specification and requirements (what the software should do) are important.
- Where appropriate, you should reuse software that has already been developed rather than write new software.

◀◀ ▶▶ ◀ ▶ ◀ Doc Doc ▶ Back Close

1.3.3. Web software engineering

- Software reuse is the dominant approach for constructing web-based systems.
  - When building these systems, you think about how you can assemble them from pre-existing software components and systems.
- Web-based systems should be developed and delivered incrementally.
  - It is now generally recognized that it is impractical to specify all the requirements for such systems in advance.
- User interfaces are constrained by the capabilities of web browsers.

- Technologies such as AJAX allow rich interfaces to be created within a web browser but are still difficult to use. Web forms with local scripting are more commonly used.

Web-based systems are complex distributed systems but the fundamental principles of software engineering discussed previously are as applicable to them as they are to any other types of system.

The fundamental ideas of software engineering, discussed in the previous section, apply to web-based software in the same way that they apply to other types of software system.

Revision Questions

EXERCISE 1. ✍ What is the difference between software engineering and system engineering?

EXERCISE 2. ✍ What are the best software engineering techniques and methods?

References and Additional Reading Materials

1. Hans van Vliet (2005). Software Engineering: Principles and Practice (2nd ed.). John Wiley & Sons, Inc. ISBN: 0-471-97508-7

2. Ian Sommerville, Pete Sawyer(2005). Requirements Engineering: A Good Practice Guide. ISBN: 0-471-97444-7. John Wiley & Sons, Inc

3. Gerald Kotonya, Ian Sommerville (1998). Requirements Engineering: Processes and Techniques. John Wiley & Sons, Inc. ISBN: 0-471-97208-8

Solutions to Exercises

Exercise 1.

System engineering is concerned with all aspects of computer-based systems development including hardware, software and process engineering. Software engineering is part of this more general process.

Exercise 1

Exercise 2.

While all software projects have to be professionally managed and developed, different techniques are appropriate for different types of system. For example, games should always be developed using a series of prototypes whereas safety critical control systems require a complete and analyzable specification to be developed. You can't, therefore, say that one method is better than another.

Exercise 2